

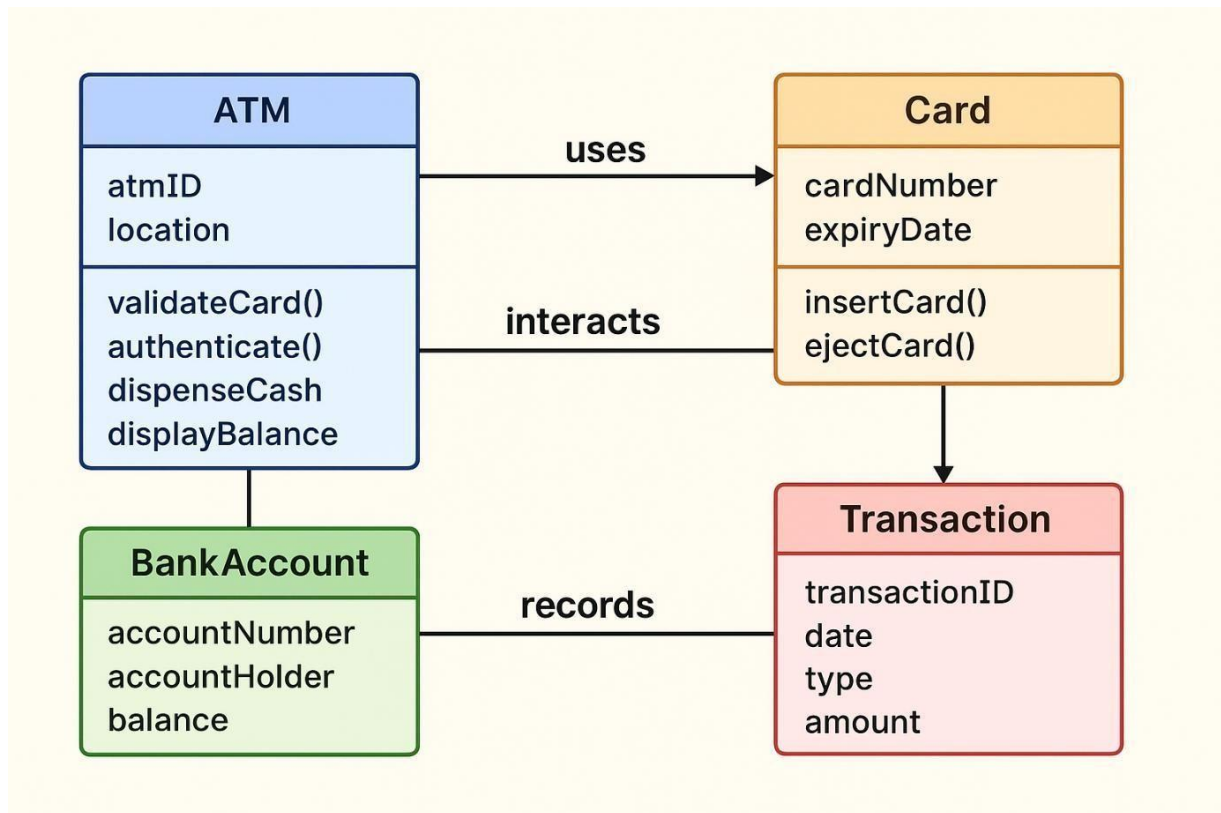
OOMD UNIT 1

Q. 1 Draw class diagram for ATM system.

5 marks

Ans:-

Class Diagram for ATM System



Classes and Relationships:

1. ATM

- Attributes: atmID, location
- Methods: insertCard(), ejectCard(), enterPin(), displayMenu()

2. BankAccount

- Attributes: accountNumber, accountHolderName, balance
- Methods: checkBalance(), withdrawAmount(), depositAmount()

3. Card

- Attributes: cardNumber, expiryDate, pin
- Methods: validatePin()

4. Transaction

- Attributes: transactionID, date, amount, type
- Methods: executeTransaction()

Relationships:

- ATM **uses** Card
- ATM **performs** Transaction
- Transaction **linked to** BankAccount
- Card **belongs to** BankAccount

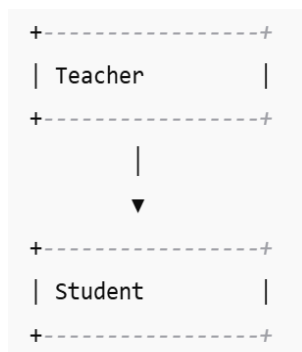
Q. 2 Explain the types of relationship along with their notations. 5 marks

Ans:-

Types of Relationships in Object-Oriented Modeling and Design (OOMD):

Association

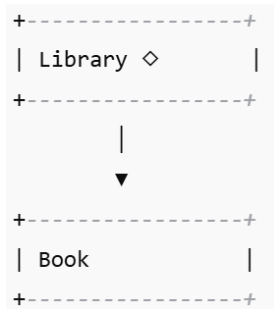
- **Definition:** An Association Class is a class attached to an association relationship between two other classes. It is used when you need to store information about the relationship itself. This is necessary when attributes belong to the link between objects, not to either of the objects themselves.
- **Notation:** Connect classes with a simple line.
- **Example:**



Aggregation (Has-A Relationship)

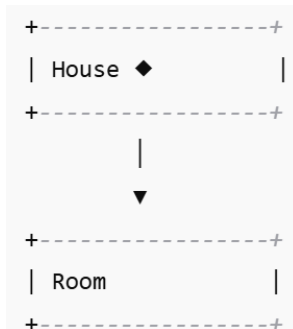
- **Definition:** One class is part of another class, but both remain independent. A special type of association representing a "part-of" relationship. The "part" can exist independently of the "whole."
- **Notation:** Line with a hollow diamond at the "whole" class side.
- **Example:**

SPPU TE BE COMP 📌 [Click to join telegram](#)



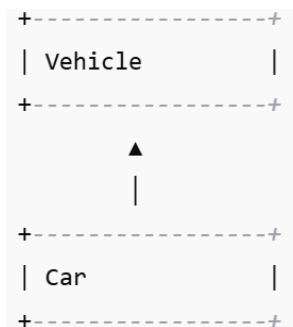
Composition (Strong Has-A Relationship)

- **Definition:** One class is part of another class and both are dependent. If the whole is deleted, the part is also deleted.
- **Notation:** Line with a filled (black) diamond at the “whole” class side.
- **Example:**



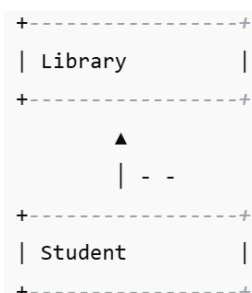
Inheritance (Is-A Relationship)

- **Definition:** When one class inherits properties and methods from another class.
- **Notation:** Line with an arrowhead pointing towards the parent class.
- **Example:**



Dependency

- **Definition:** When one class temporarily uses another class. One class is dependent on another class.
- **Notation:** Dashed arrow pointing from the dependent class.
- **Example:**



Q. 3 Define object? Explain components of object diagram in brief?

5 marks

Ans:-

Definition of Object:

An **object** is a real-world entity or concept that has **attributes (data)** and **behaviors (methods/functions)**. It represents a **specific instance of a class**, and each object has its own **state**.

Components of Object Diagram (in brief):

1. Object Name:

- Shows the name of the object and its class.
- Notation: `objectName : ClassName`
- Example: `atm1 : ATM`

2. Attributes:

- Represents the properties or characteristics of the object.
- Example: `balance = 5000, status = active`

3. Links/Relationships:

- Shows the connection or association between objects.
- Example: `customer1 — atm1`

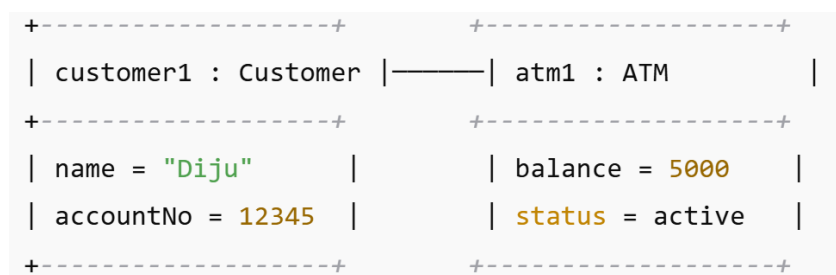
4. Values:

- Shows the current values of an object's attributes to represent its **state at a particular time**.
- Example: `pin = 1234, withdrawalLimit = 10000`

5. Multiplicity (Optional in brief):

- Shows how many instances of an object can be associated with another object.
- Example: `1..*` for one-to-many relationship

Mini Object Diagram Example



Explanation for Notebook:

- Rectangle top → objectName : ClassName
- Rectangle middle → Attributes with current values
- Line → Link / relationship between objects

Q. 4 Difference between link and association with suitable example.

5 marks

Ans:-

Aspect	Association	Link
Definition	A logical relationship between two or more classes in a class diagram.	A real connection between two objects in an object diagram.
Represents	Conceptual connection (class to class).	Actual instance connection (object to object).
Level	Works at class level (blueprint / design).	Works at object level (instance / runtime).
Nature	Static relationship, exists in design phase.	Dynamic relationship, exists during runtime.
Multiplicity	Shows how many objects of a class can be associated with another class. Optional in design.	Can show number of object instances in a relationship, optional.
Notation	Solid line between classes in class diagram.	Solid line between objects in object diagram.
Example	Customer — ATM (Customer class associated with ATM class)	customer1 — atm1 (customer1 object linked with atm1 object)
Use / Additional Notes	Useful during design phase to plan interactions between classes.	Useful during runtime or scenario analysis to understand real object behavior.

Q. 5 what is OO development? Explain the OO themes?

5 marks

Ans:-

Object-Oriented (OO) Development

Definition:

Object-Oriented Development is a **software development approach** that uses **objects** as the **basic building blocks** of a system.

- Each object has **attributes (data)** and **behaviors (methods/functions)**.
- Focuses on **reusability, modularity, and modeling real-world entities**.
- Example: A Car object can have attributes like color and behaviors like drive().

OO Themes / Concepts

1. Objects:

- Real-world entities represented in software.
 - Have **attributes and behaviors**.
 - Example: atm1 : ATM with balance and withdraw() method.
2. **Classes:**
- Blueprint or template for creating objects.
 - Example: class ATM defines balance, withdraw(), deposit().
3. **Inheritance:**
- A class **inherits properties and methods** from another class.
 - Example: SavingsAccount inherits from Account.
4. **Encapsulation:**
- Wrapping **data and methods** into a single unit (class).
 - Hides internal details.
5. **Polymorphism:**
- A method can **behave differently** in different objects.
 - Example: withdraw() in SavingsAccount vs CurrentAccount.
6. **Abstraction:**
- Hiding unnecessary details, showing only essential features.
 - Example: User sees ATM.withdraw() but not internal processing.

Q. 6 Explain the following terms with respect to object oriented methodology. 5 marks

1. Abstraction

2. Encapsulation

3. Sharing

Ans:-

1. Abstraction

Definition:

Abstraction is the principle of hiding complex implementation details from the user and showing only the essential, relevant features of an object. It simplifies the interaction with an object by providing a high-level view and reducing complexity. The main goal of abstraction is to manage the complexity of large systems.

Key Points:

- Focuses on **what an object does**, not **how it does it**.
- Reduces system complexity and supports modularity.

Example:

- In an ATM, the user uses `withdraw()` method.
- Internal steps like verifying account, updating balance are **hidden from the user**.

2. Encapsulation

Definition:

Encapsulation is the mechanism of bundling the data (attributes) and the methods (operations) that operate on that data into a single unit, known as a class. It also involves restricting direct access to an object's internal state; this is called data hiding.

Key Points:

- Protects **internal data** from unauthorized access.
- Provides **controlled access** through methods (getters/setters).

Example:

- In a `BankAccount` class, `balance` is private.
- Users access it through methods like `deposit()` or `withdraw()` only.

3. Sharing

Definition:

Sharing means **reusing existing objects, classes, or methods** in multiple places to reduce duplication and save resources. In the context of OO methodology, refers to the concept of inheritance. It is a mechanism where a new class (a subclass or child class) can derive, or inherit, attributes and methods from an existing class (a superclass or parent class). This promotes code reuse and establishes a clear hierarchical relationship between classes.

Key Points:

- Helps in **efficient resource utilization**.
- Supports **modularity and maintainability**.

Example:

- A `Printer` object shared by multiple `Computer` objects in a network.
- Multiple objects can **use the same methods or resources** without creating new copies.

Q. 7 Explain the element of class diagram with examples.

6 marks

Ans:-

Elements of Class Diagram

Definition:

Class diagram is a **static structure diagram** in Object-Oriented Modeling that shows **classes, their attributes, methods, and relationships**.

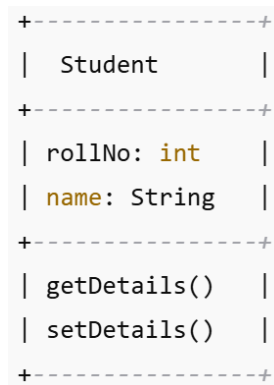
Elements

1. **Class:**

SPPU TE BE COMP Click to join telegram

- Blueprint for objects
- Rectangle divided into 3 compartments: Class Name | Attributes | Methods

○ **Example:**



• **Attributes:**

- Properties or data members of a class
- Show state of object
- Example: rollNo, name

• **Methods / Operations:**

- Functions or behaviors of class
- Show what object can do
- Example: getDetails(), setDetails()

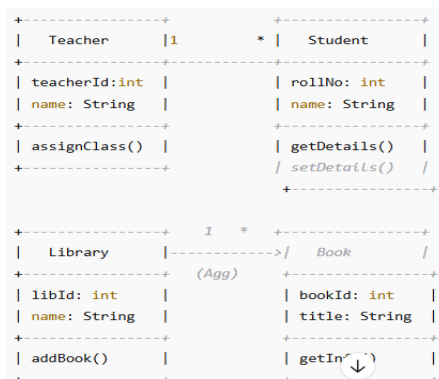
• **Relationships / Links:**

- **Association:** Teacher — Student (general connection)
- **Aggregation (Has-A):** Library — Book
- **Composition:** House — Room (strong ownership)
- **Inheritance / Generalization:** SavingsAccount inherits Account

• **Multiplicity (Optional):**

- Number of objects involved in a relationship
- Example: 1 . . * one object connected to many objects

Class Diagram Showing Classes, Attributes, Methods, and Relationships



Q. 8 What do you mean by Association class?

4 marks

Ans:-

Association Class

Definition:

An **Association Class** is a **class that connects two other classes** and **stores information about the relationship** between them.

- It is used when a relationship itself has **attributes or operations**.

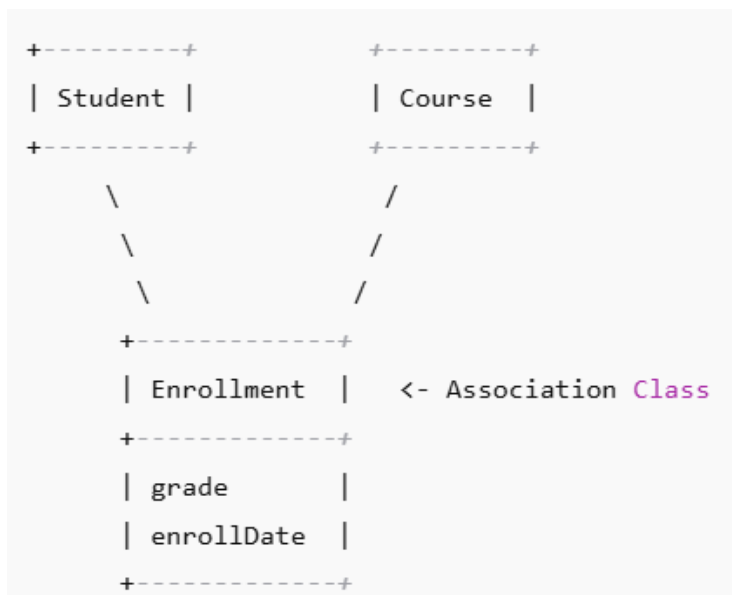
Key Points:

1. Shows **relationship + extra data** about the relationship.
2. Combines **association and class** in a single element.

Example:

- Student — Course (association)
- Association class: Enrollment stores **grade, enrollmentDate**

Association Class Diagram



Q. 9 Explain different relations in UML.

5 marks

Ans:- refer Q.2 as it is.

Q. 10 What are different type of analysis classes? Explain with an example.

6 mar

Ans:-

Different Types of Analysis Classes

Definition:

Analysis classes are **classes identified during the analysis phase** of Object-Oriented Development. They **represent concepts, objects, or events** in the problem domain.

Types of Analysis Classes:

1. Entity Classes

- **Definition:** Represent **real-world objects or things** with attributes and behaviors.
- **Example:** Student, Book, Car
- Typically **nouns** in the problem description.

2. Boundary Classes (Interface Classes)

- **Definition:** Represent **interaction between system and actors** (users or external systems).
- **Example:** ATM Screen, Login Form, Payment Gateway
- Handles **input/output operations**.

3. Control Classes

- **Definition:** Represent **processing or control logic** between entity and boundary classes.
- **Example:** TransactionManager, OrderProcessor
- Handles **business rules and workflow**.

Example in ATM System

Class Type	Example	Role
Entity Class	Account	Stores account data (balance, accountNo)
Boundary Class	ATM Screen	Handles user input/output
Control Class	TransactionManager	Processes withdrawals, deposits, and updates

ATM System – Analysis Classes Diagram

[Boundary Class]		[Control Class]		[Entity Class]
+-----+		+-----+		+-----+
ATM Screen	---->	TransactionManager		Account
+-----+		+-----+		+-----+
display()		processWithdrawal()		accountNo
getInput()		processDeposit()		balance
+-----+		+-----+		+-----+

Q. 11 How you model class & object in UML.

4 marks

Ans:-

Modeling Class & Object in UML

1. Class Modeling

- **Definition:** A class represents a **blueprint of objects** with common attributes and methods.
- **Notation:** Rectangle divided into 3 compartments:
 1. **Class Name** (top)
 2. **Attributes** (middle)
 3. **Methods / Operations** (bottom)
- **Example:**

```
+-----+
|   Student   |
+-----+
| rollNo: int  |
| name: String |
+-----+
| getDetails() |
| setDetails() |
+-----+
```

2. Object Modeling

- **Definition:** An object is an **instance of a class**, representing a **real entity**.
- **Notation:** Rectangle with **objectName : ClassName** at the top. Attributes can be shown with current values.
- **Example:**

```
+-----+
| s1 : Student |
+-----+
| rollNo = 101  |
| name = "Ravi" |
+-----+
```

UNIT 2

Q. 1 Draw state diagram with composite states from ATM card reading & authentication system.

5 marks

Ans:-

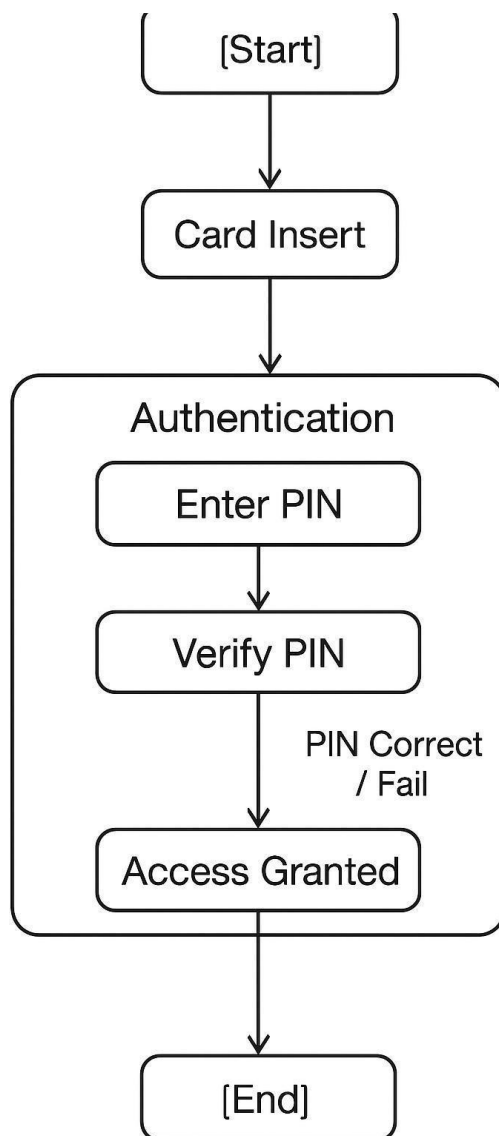
State Diagram – ATM Card Reading & Authentication

Definition:

A **state diagram** shows the **states of an object** and **transitions triggered by events**.

- **Composite state:** A state that contains **nested sub-states**, showing detailed behavior inside

ATM Example – Composite State Diagram



Q. 2 Draw state chart diagram for Railway reservation system.

5 marsk

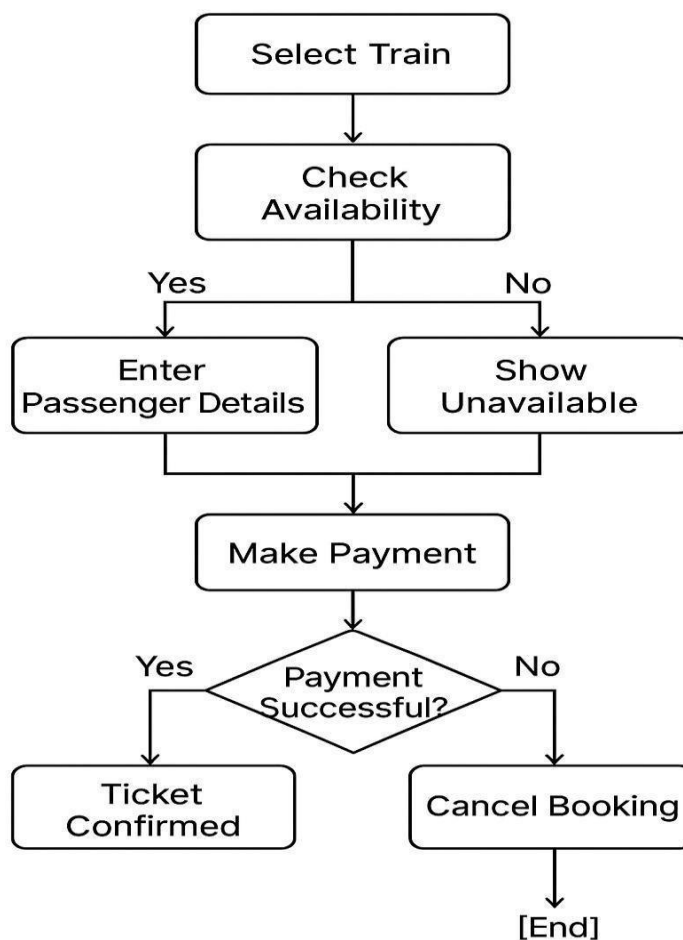
Ans:-

Definition:

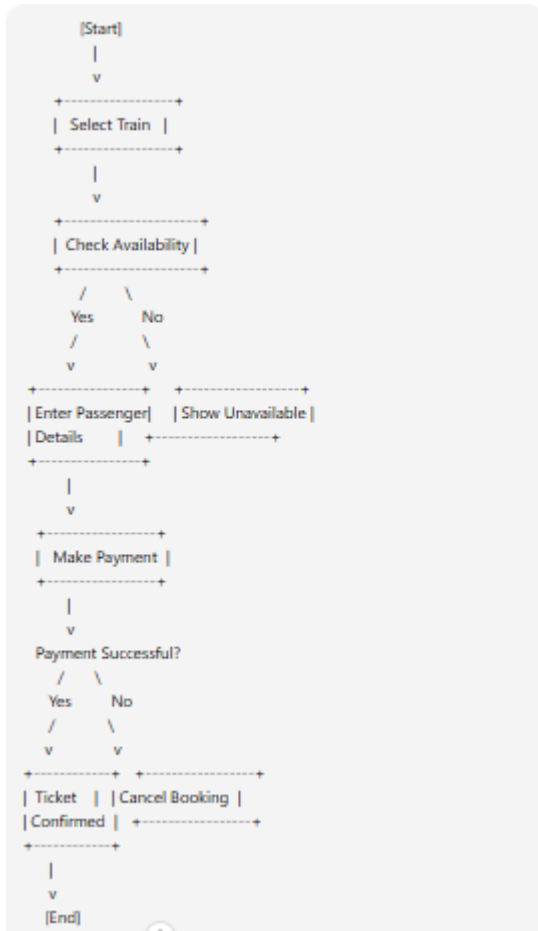
A state chart diagram shows **different states of an object/system** and **transitions caused by events**.

Useful for **dynamic behavior modeling** of a system.

Railway Reservation System – State Chart Diagram



And refer this



Q. 3 Define event and explain in brief different types of events.

5 marks

Ans:-

Event – Definition & Types

Definition:

- An **event** is an **occurrence or happening that triggers a change of state** in a system or object.
- Events represent **stimuli that the system responds to**, causing transitions between states in a state diagram.
- Example: When a user presses a button, when a timer expires, or when a signal/message is received.

Types of Events

1. External Event:

- Originates **outside the system** and is triggered by an actor (user or external system).
- **Example:** A user clicks the “Submit” button in a form.

- **Explanation:** External events help the system interact with its environment and initiate processing.
- 2. **Internal Event:**
 - Generated **within the system** itself, usually due to some internal condition or activity.
 - **Example:** Timer expires, threshold reached, or a process completes.
 - **Explanation:** Internal events are important for automated system behavior without user intervention.
- 3. **Time Event:**
 - Triggered **after a specified time** or at a specific moment.
 - **Example:** Alarm goes off after 5 minutes, or a reservation expires after 24 hours.
 - **Explanation:** Time events are useful for scheduling, delays, or periodic checks.
- 4. **Signal Event:**
 - Caused when the system **receives a message or signal**.
 - **Example:** Notification received from another system, or incoming data triggers a state change.
 - **Explanation:** Signal events allow asynchronous communication between objects or systems.
- 5. **Change Event:**
 - Occurs **when a condition or variable changes** its value, causing a state transition.
 - **Example:** Stock level falls below the minimum, or account balance becomes zero.
 - **Explanation:** Change events monitor conditions continuously and trigger actions automatically.

Q. 4 Explain how do you model different events in state diagram

5 marks

Ans:-

Modeling Events in State Diagram

Definition:

- In a **state diagram**, an event is modeled as a **trigger for a transition** between states.
- Events **cause the system/object to change its state**.
- Each arrow in the diagram usually represents **an event causing the transition**.

How to Model Events

1. **Identify States:**
 - List all the states of the object/system first.
 - Example: Idle, Card Inserted, Authentication, Access Granted (for ATM).
2. **Identify Events:**
 - Determine which events **trigger transitions** between states.
 - Example: Insert Card, Enter PIN, Correct PIN, Incorrect PIN.
3. **Draw Transitions:**

SPPU TE BE COMP [Click to join telegram](#)

- Represent events as **arrows from source state to target state**.
- Use notation:

Source State --> Target State : Event / Action

- **Event** → triggers the transition
 - **Action** (optional) → performed after the event
4. **Decision Points (Optional):**
- For multiple possible outcomes of an event, show **Yes/No** or **condition-based branches**.
 - Example: PIN correct → Access Granted, PIN incorrect → Retry.
5. **Label Arrows Clearly:**
- Always write **event names on arrows**
 - Actions can be written after “/” if needed

Example – ATM System

[Idle] -- Insert Card --> [Card Inserted]

[Card Inserted] -- Enter PIN --> [Authentication]

[Authentication] -- PIN Correct --> [Access Granted]

[Authentication] -- PIN Incorrect --> [Retry / Cancel]

Note- if you have to draw diagram so refer above atm diagram.

Q. 5 Write short note on:

5 marks

1. . Package
2. Derived data
3. Constarints

Ans:-

Package

- **Definition:**
A package in UML is a grouping mechanism used to organize elements of a model, such as classes, interfaces, and use cases, into related groups. It is visually represented as a file folder symbol. Packages are essential for managing the complexity of large-scale systems by dividing them into smaller, more manageable and understandable subsystems.
- **Purpose:**
 - Helps **organize large systems**
 - Provides **modularity and structure**
- **Notation:**
 - Rectangle with a small tab at the top (like a folder)
- **Example:**

SPPU TE BE COMP Click to join telegram

- <<Package>> Banking containing classes Account, Customer, Transaction.

Derived Data

- **Definition:**
A derived element, such as a derived attribute or association, is one whose value can be computed or inferred from other information present in the model. It does not represent a new piece of information but rather a value that can be calculated. In a class diagram, a derived attribute is denoted by a forward slash / before its name.
- **Purpose:**
 - Avoids data redundancy
 - Ensures data **consistency and accuracy**
- **Notation:**
 - Use a **forward slash** / before the attribute name in UML
- **Example:**
 - /totalAmount = sum of all transaction amounts
 - /age = current year – birth year

Constraints

- **Definition:**
A constraint is a rule or condition that specifies a restriction on a UML model element. It must always be maintained as true. Constraints are written in plain text and enclosed in curly braces {}. They are used to add rules to the model that cannot be expressed by the standard UML notations.
- **Purpose:**
 - Ensures **validity of data**
 - Maintains **business rules or system rules**
- **Notation:**
 - Written inside **curly braces {}**
- **Example:**
 - {balance >= 0} → Account balance cannot be negative
 - {age >= 18} → Customer must be adult

Q. 6 What do you mean by Association ends? Explain N-ary association in short.

5 marks

Ans:-

Association Ends

- **Definition:**
 - **Association ends** represent the **roles played by objects of classes in a relationship**.
 - They define **how one class is connected to another**.
- **Features:**
 - Can have a **name (role)**
 - Can specify **multiplicity** (1, 0..*, etc.)
 - Can show **navigability** (unidirectional or bidirectional)
- **Example:**
 - Teacher — Student

SPPU TE BE COMP Click to join telegram

- Teacher side → role: teaches
- Student side → role: learns

- Multiplicity: 1 teacher teaches many students → 1..*

N-ary Association (Short)

- **Definition:**
 - An **N-ary association** is a relationship **involving more than two classes**.
 - Used when a simple binary association is not enough to describe the relationship.
- **Example:**
 - Classes: Student, Course, Instructor
 - N-ary association: Enrollment connecting **all three classes**
 - Shows **which student is enrolled in which course with which instructor**
- **Notation:**
 - Draw a **diamond in the center**, connect **all participating classes** to it

Q. 7 What is state? Explain with example transitions and condition.

5 marks

Ans: -

State – Definition & Explanation

Definition:

- A **state** is a **condition or situation of an object/system at a particular time**, during which it **satisfies some condition, performs an activity, or waits for an event**.
- States show **how an object behaves over time**.

Key Concepts

Transition:

1. Transition:

- A **transition** is a **change from one state to another**.
- It happens when an **event occurs** that causes the object/system to move to a new state.
- **Notation:** Arrow from source state → target state
- **Example:**

[Idle] -- Insert Card --> [Card Inserted]

- **Explanation:** The system moves from Idle state to Card Inserted state when the user inserts a card.

2. Event / Trigger:

- The **event** is the **occurrence that initiates the transition**.
- **Example:** Insert Card triggers the transition from Idle → Card Inserted.
- **Explanation:** Events are the “stimuli” that the system responds to.

SPPU TE BE COMP  Click to join telegram

3. Condition / Guard:

SPPU TE BE COMP [Click to join telegram](#)

- A **condition or guard** is a **boolean expression that must be true** for the transition to occur.
- **Notation:** Written in square brackets [] on the transition.
- **Example:**

[PIN Correct] --> [Access Granted]
[PIN Incorrect] --> [Retry / Cancel]

- **Explanation:** Even if the event occurs, the transition only happens when the **condition is satisfied**.

Example – ATM System State Diagram

[Idle] -- Insert Card --> [Card Inserted]

[Card Inserted] -- Enter PIN --> [Authentication]

[Authentication] -- [PIN Correct] --> [Access Granted]

[Authentication] -- [PIN Incorrect] --> [Retry / Cancel]

Q. 8 Explain following:

5 marks

1. Composite state

2. Self transition orthogonal states with respect to state chart diagram.

Ans:-

Composite State

- **Definition:**
 - A **composite state** is a **state that contains one or more nested sub-states** inside it.
 - It is used to **group related states together** and show **complex behavior hierarchically**.
- **Purpose:**
 - Simplifies large state diagrams
 - Represents **hierarchical behavior**
- **Example:**
 - Authentication in ATM system is a composite state containing:
 - Enter PIN
 - Verify PIN
- **Notation/ Diagram:**


```
+-----+
| Authentication | <-- Composite State
| +-----+ /
| | Enter PIN   | | <-- Sub-state
| +-----+ /
| | Verify PIN  | | <-- Sub-state
| +-----+ /
+-----+
```

Self-Transition

- **Definition:**
 - A **self-transition** is a transition where a **state transitions to itself** in response to an event.
- **Purpose:**
 - Represents an **internal change or repeated activity** without leaving the state
- **Example:**
 - In Idle state, pressing **Cancel** repeatedly → stays in Idle
- **Notation/ Diagram :**

```
+-----+
| Idle   |
|        |
| ⤴ Event | <-- Loops back to itself
+-----+
```

Orthogonal States

- **Definition:**
 - **Orthogonal states** are **independent regions within a composite state** that can be **active simultaneously**.
 - Used to model **parallel or concurrent behavior**
- **Example:**
 - In Railway Reservation System:
 - Region 1 → Seat Selection
 - Region 2 → Payment Processing
 - Both regions active at the same time inside a composite state Reservation
- **Notation/ Diagram:**

```
+-----+
| Reservation | <-- Composite State
| +-----+ +-----+ /
| | Seat      | | Payment | | <-- Orthogonal / parallel regions
| | Selection | | Processing|
| +-----+ +-----+ /
+-----+
```

Q. 9 Explain terms:

5 marks

1. Aggregation
2. Composition

Ans:-

Aggregation

- **Definition:**
 - Aggregation is a **“Has-A” relationship** where one class **contains another class** as a part, but **both can exist independently**.
 - It represents a **weak ownership**.
- **Notation:**
 - A **hollow diamond** at the end of the association line near the **whole** class.
- **Example:**
 - Library aggregates Book →
 - Library has many books
 - Books can exist even if Library is deleted
- **Diagram:**

Library ◇—— Book

Composition

- **Definition:**
 - Composition is a **strong “Has-A” relationship** where one class **owns another class**, and **the part cannot exist without the whole**.
 - It represents **strong ownership**.
- **Notation:**
 - A **filled diamond** at the end of the association line near the **whole** class.
- **Example:**
 - House composes Room →
 - Rooms cannot exist without the House
 - If House is deleted, all Rooms are deleted too
- **Diagram :**

House ◆—— Room

Q. 10 Explain different components of a state diagram

5 marks

Ans:-

Components of a State Diagram

A **state diagram** (or state chart diagram) is used to **model the dynamic behavior of a system or object**, showing **how it moves from one state to another in response to events**.

The main components are:

State

- Represents a **condition or situation of an object** during which it **performs activities or waits for events**.
- **Example:** Idle, Authentication, Access Granted in an ATM system.

Initial State

- The **starting point** of the state diagram.
- **Notation:** A **filled black circle**.
- **Example:** System starts in Idle state.

Final State

- Represents the **end point** of the process or object lifecycle.
- **Notation:** A **filled black circle inside a hollow circle**.
- **Example:** ATM transaction ends after Access Granted or Cancel.

Transition

- **Change from one state to another** triggered by an **event**.
- **Notation:** Arrow from source state → target state.
- **Example:** [Idle] -- Insert Card --> [Card Inserted]

Event / Trigger

- **Stimulus that causes a transition** between states.
- **Example:** Insert Card, Enter PIN, PIN Correct.

Condition / Guard

- A **boolean expression** that must be true for a transition to occur.
- **Notation:** Written in square brackets [].
- **Example:** [PIN Correct] → Access Granted

Action (Optional)

- **Activity performed** as a result of a transition.
- **Notation:** Written after “/” on the transition.
- **Example:** [PIN Correct] / Dispense Cash

Definition:

Derived data is information that is **calculated or obtained from other data** rather than stored directly.

- **Purpose:**
 - Avoids data redundancy
 - Ensures data **consistency and accuracy**
- **Notation:**
 - Use a **forward slash /** before the attribute name in UML
- **Example:**
 - /totalAmount = sum of all transaction amounts
 - /age = current year – birth year

Constraints

- **Definition:**

A **constraint** is a **rule or restriction** applied to a class, attribute, or relationship in UML.
- **Purpose:**
 - Ensures **validity of data**
 - Maintains **business rules or system rules**
- **Notation:**
 - Written inside **curly braces {}**
- **Example:**
 - {balance >= 0} → Account balance cannot be negative
 - {age >= 18} → Customer must be adult

Q. 6 What do you mean by Association ends? Explain N-ary association in short. 5 marks

Ans:-

Association Ends

- **Definition:**
 - **Association ends** represent the **roles played by objects of classes in a relationship**.
 - They define **how one class is connected to another**.
- **Features:**
 - Can have a **name (role)**
 - Can specify **multiplicity** (1, 0..*, etc.)
 - Can show **navigability** (unidirectional or bidirectional)
- **Example:**
 - Teacher — Student
 - Teacher side → role: teaches
 - Student side → role: learns
 - Multiplicity: 1 teacher teaches many students → 1..*

N-ary Association (Short)

- **Definition:**
 - An **N-ary association** is a relationship **involving more than two classes**.
 - Used when a simple binary association is not enough to describe the relationship.
- **Example:**
 - Classes: Student, Course, Instructor
 - N-ary association: Enrollment connecting **all three classes**
 - Shows **which student is enrolled in which course with which instructor**
- **Notation:**
 - Draw a **diamond in the center**, connect **all participating classes** to it

Q. 7 What is state? Explain with example transitions and condition.

5 marks

Ans: -

State – Definition & Explanation

Definition:

- A **state** is a **condition or situation of an object/system at a particular time**, during which it **satisfies some condition, performs an activity, or waits for an event**.
- **States show how an object behaves over time.**

Key Concepts

Transition:

4. Transition:

- A **transition** is a **change from one state to another**.
- It happens when an **event occurs** that causes the object/system to move to a new state.
- **Notation:** Arrow from source state → target state
- **Example:**

[Idle] -- Insert Card --> [Card Inserted]

- **Explanation:** The system moves from Idle state to Card Inserted state when the user inserts a card.

5. Event / Trigger:

- The **event** is the **occurrence that initiates the transition**.
- **Example:** Insert Card triggers the transition from Idle → Card Inserted.
- **Explanation:** Events are the “stimuli” that the system responds to.

6. Condition / Guard:

SPPU TE BE COMP Click to join telegram

- A **condition or guard** is a **boolean expression that must be true** for the transition to occur.
- **Notation:** Written in square brackets [] on the transition.
- **Example:**

[PIN Correct] --> [Access Granted]

[PIN Incorrect] --> [Retry / Cancel]

- **Explanation:** Even if the event occurs, the transition only happens when the **condition is satisfied**.

Example – ATM System State Diagram

[Idle] -- Insert Card --> [Card Inserted]

[Card Inserted] -- Enter PIN --> [Authentication]

[Authentication] -- [PIN Correct] --> [Access Granted]

[Authentication] -- [PIN Incorrect] --> [Retry / Cancel]

Q. 8 Explain following:

5 marks

3. Composite state

4. Self transition orthogonal states with respect to state chart diagram.

Ans:-

Composite State

- **Definition:**
 - A **composite state** is a **state that contains one or more nested sub-states** inside it.
 - It is used to **group related states together** and show **complex behavior hierarchically**.
- **Purpose:**
 - Simplifies large state diagrams
 - Represents **hierarchical behavior**
- **Example:**
 - Authentication in ATM system is a composite state containing:
 - Enter PIN
 - Verify PIN
- **Notation/ Diagram:**

```
+-----+
| Authentication | <-- Composite State
| +-----+ /
| | Enter PIN   | | <-- Sub-state
| +-----+ /
| | Verify PIN  | | <-- Sub-state
| +-----+ /
+-----+
```

Self-Transition

- **Definition:**
 - A **self-transition** is a transition where a **state transitions to itself** in response to an event.
- **Purpose:**
 - Represents an **internal change or repeated activity** without leaving the state
- **Example:**
 - In Idle state, pressing **Cancel** repeatedly → stays in Idle
- **Notation/ Diagram :**

```
+-----+
| Idle   |
|        |
| ⤿ Event| <-- Loops back to itself
+-----+
```

Orthogonal States

- **Definition:**
 - **Orthogonal states** are **independent regions within a composite state** that can be **active simultaneously**.
 - Used to model **parallel or concurrent behavior**
- **Example:**
 - In Railway Reservation System:
 - Region 1 → Seat Selection
 - Region 2 → Payment Processing
 - Both regions active at the same time inside a composite state Reservation
- **Notation/ Diagram:**

```
+-----+
| Reservation | <-- Composite State
| +-----+ +-----+ /
| | Seat      | | Payment | | <-- Orthogonal / parallel regions
| | Selection | | Processing|
| +-----+ +-----+ /
+-----+
```

Q. 9 Explain terms:

5 marks

3. Aggregation

4. Composition

Ans:-

Aggregation

- **Definition:**
 - Aggregation is a **“Has-A” relationship** where one class **contains another class** as a part, but **both can exist independently**.
 - It represents a **weak ownership**.
- **Notation:**
 - A **hollow diamond** at the end of the association line near the **whole** class.
- **Example:**
 - Library aggregates Book →
 - Library has many books
 - Books can exist even if Library is deleted
- **Diagram:**

Library ◇—— Book

Composition

- **Definition:**
 - Composition is a **strong “Has-A” relationship** where one class **owns another class**, and **the part cannot exist without the whole**.
 - It represents **strong ownership**.
- **Notation:**
 - A **filled diamond** at the end of the association line near the **whole** class.
- **Example:**
 - House composes Room →
 - Rooms cannot exist without the House
 - If House is deleted, all Rooms are deleted too
- **Diagram :**

House ◆—— Room

Q. 10 Explain different components of a state diagram

5 marks

Ans:-

Components of a State Diagram

A **state diagram** (or state chart diagram) is used to **model the dynamic behavior of a system or object**, showing **how it moves from one state to another in response to events**.

The main components are:

State

- Represents a **condition or situation of an object** during which it **performs activities or waits for events**.
- **Example:** Idle, Authentication, Access Granted in an ATM system.

Initial State

- The **starting point** of the state diagram.
- **Notation:** A **filled black circle**.
- **Example:** System starts in Idle state.

Final State

- Represents the **end point** of the process or object lifecycle.
- **Notation:** A **filled black circle inside a hollow circle**.
- **Example:** ATM transaction ends after Access Granted or Cancel.

Transition

- **Change from one state to another** triggered by an **event**.
- **Notation:** Arrow from source state → target state.
- **Example:** [Idle] -- Insert Card --> [Card Inserted]

Event / Trigger

- **Stimulus that causes a transition** between states.
- **Example:** Insert Card, Enter PIN, PIN Correct.

Condition / Guard

- A **boolean expression** that must be true for a transition to occur.
- **Notation:** Written in square brackets [].
- **Example:** [PIN Correct] → Access Granted

Action (Optional)

- **Activity performed** as a result of a transition.
- **Notation:** Written after “/” on the transition.

Example: [PIN Correct] / Display

- **Definition:**
 - An **N-ary association** is a relationship **involving more than two classes**.
 - Used when a simple binary association is not enough to describe the relationship.
- **Example:**
 - Classes: Student, Course, Instructor
 - N-ary association: Enrollment connecting **all three classes**
 - Shows **which student is enrolled in which course with which instructor**
- **Notation:**
 - Draw a **diamond in the center**, connect **all participating classes** to it

Q. 7 What is state? Explain with example transitions and condition.

5 marks

Ans: -

State – Definition & Explanation

Definition:

- A **state** is a **condition or situation of an object/system at a particular time**, during which it **satisfies some condition, performs an activity, or waits for an event**.
- **States show how an object behaves over time.**

Key Concepts

Transition:

7. Transition:

- A **transition** is a **change from one state to another**.
- It happens when an **event occurs** that causes the object/system to move to a new state.
- **Notation:** Arrow from source state → target state
- **Example:**

[Idle] -- Insert Card --> [Card Inserted]

- **Explanation:** The system moves from Idle state to Card Inserted state when the user inserts a card.

8. Event / Trigger:

- The **event** is the **occurrence that initiates the transition**.
- **Example:** Insert Card triggers the transition from Idle → Card Inserted.
- **Explanation:** Events are the “stimuli” that the system responds to.

9. Condition / Guard:

SPPU TE BE COMP Click to join telegram

- A **condition or guard** is a **boolean expression that must be true** for the transition to occur.
- **Notation:** Written in square brackets [] on the transition.
- **Example:**

[PIN Correct] --> [Access Granted]

[PIN Incorrect] --> [Retry / Cancel]

- **Explanation:** Even if the event occurs, the transition only happens when the **condition is satisfied**.

Example – ATM System State Diagram

[Idle] -- Insert Card --> [Card Inserted]

[Card Inserted] -- Enter PIN --> [Authentication]

[Authentication] -- [PIN Correct] --> [Access Granted]

[Authentication] -- [PIN Incorrect] --> [Retry / Cancel]

Q. 8 Explain following:

5 marks

5. Composite state

6. Self transition orthogonal states with respect to state chart diagram.

Ans:-

Composite State

- **Definition:**
 - A **composite state** is a **state that contains one or more nested sub-states** inside it.
 - It is used to **group related states together** and show **complex behavior hierarchically**.
- **Purpose:**
 - Simplifies large state diagrams
 - Represents **hierarchical behavior**
- **Example:**
 - Authentication in ATM system is a composite state containing:
 - Enter PIN
 - Verify PIN
- **Notation/ Diagram:**

```
+-----+
| Authentication | <-- Composite State
| +-----+ /
| | Enter PIN   | | <-- Sub-state
| +-----+ /
| | Verify PIN  | | <-- Sub-state
| +-----+ /
+-----+
```

Self-Transition

- **Definition:**
 - A **self-transition** is a transition where a **state transitions to itself** in response to an event.
- **Purpose:**
 - Represents an **internal change or repeated activity** without leaving the state
- **Example:**
 - In Idle state, pressing **Cancel** repeatedly → stays in Idle
- **Notation/ Diagram :**

```
+-----+
| Idle   |
|        |
| ⤴ Event | <-- Loops back to itself
+-----+
```

Orthogonal States

- **Definition:**
 - **Orthogonal states** are **independent regions within a composite state** that can be **active simultaneously**.
 - Used to model **parallel or concurrent behavior**
- **Example:**
 - In Railway Reservation System:
 - Region 1 → Seat Selection
 - Region 2 → Payment Processing
 - Both regions active at the same time inside a composite state Reservation
- **Notation/ Diagram:**

```
+-----+
| Reservation | <-- Composite State
| +-----+ +-----+ /
| | Seat      | | Payment | | <-- Orthogonal / parallel regions
| | Selection | | Processing|
| +-----+ +-----+ /
+-----+
```

Q. 9 Explain terms:

5 marks

5. Aggregation

6. Composition

Ans:-

Aggregation

- **Definition:**
 - Aggregation is a **“Has-A” relationship** where one class **contains another class** as a part, but **both can exist independently**.
 - It represents a **weak ownership**.
- **Notation:**
 - A **hollow diamond** at the end of the association line near the **whole** class.
- **Example:**
 - Library aggregates Book →
 - Library has many books
 - Books can exist even if Library is deleted
- **Diagram:**

Library ◇—— Book

Composition

- **Definition:**
 - Composition is a **strong “Has-A” relationship** where one class **owns another class**, and **the part cannot exist without the whole**.
 - It represents **strong ownership**.
- **Notation:**
 - A **filled diamond** at the end of the association line near the **whole** class.
- **Example:**
 - House composes Room →
 - Rooms cannot exist without the House
 - If House is deleted, all Rooms are deleted too
- **Diagram :**

House ◆—— Room

Q. 10 Explain different components of a state diagram

5 marks

Ans:-

Components of a State Diagram

A **state diagram** (or state chart diagram) is used to **model the dynamic behavior of a system or object**, showing **how it moves from one state to another in response to events**.

The main components are:

State

- Represents a **condition or situation of an object** during which it **performs activities or waits for events**.
- **Example:** Idle, Authentication, Access Granted in an ATM system.

Initial State

- The **starting point** of the state diagram.
- **Notation:** A **filled black circle**.
- **Example:** System starts in Idle state.

Final State

- Represents the **end point** of the process or object lifecycle.
- **Notation:** A **filled black circle inside a hollow circle**.
- **Example:** ATM transaction ends after Access Granted or Cancel.

Transition

- **Change from one state to another** triggered by an **event**.
- **Notation:** Arrow from source state → target state.
- **Example:** [Idle] -- Insert Card --> [Card Inserted]

Event / Trigger

- **Stimulus that causes a transition** between states.
- **Example:** Insert Card, Enter PIN, PIN Correct.

Condition / Guard

- A **boolean expression** that must be true for a transition to occur.
- **Notation:** Written in square brackets [].
- **Example:** [PIN Correct] → Access Granted

Action (Optional)

- **Activity performed** as a result of a transition.
- **Notation:** Written after “/” on the transition.
- **Example:** [PIN Correct] / Dispense Cash

Definition:

Derived data is information that is **calculated or obtained from other data** rather than stored directly.

- **Purpose:**
 - Avoids data redundancy
 - Ensures data **consistency and accuracy**
- **Notation:**
 - Use a **forward slash /** before the attribute name in UML
- **Example:**
 - /totalAmount = sum of all transaction amounts
 - /age = current year – birth year

Constraints

- **Definition:**

A **constraint** is a **rule or restriction** applied to a class, attribute, or relationship in UML.
- **Purpose:**
 - Ensures **validity of data**
 - Maintains **business rules or system rules**
- **Notation:**
 - Written inside **curly braces {}**
- **Example:**
 - {balance >= 0} → Account balance cannot be negative
 - {age >= 18} → Customer must be adult

Q. 6 What do you mean by Association ends? Explain N-ary association in short. 5 marks

Ans:-

Association Ends

- **Definition:**
 - **Association ends** represent the **roles played by objects of classes in a relationship**.
 - They define **how one class is connected to another**.
- **Features:**
 - Can have a **name (role)**
 - Can specify **multiplicity** (1, 0..*, etc.)
 - Can show **navigability** (unidirectional or bidirectional)
- **Example:**
 - Teacher — Student
 - Teacher side → role: teaches
 - Student side → role: learns
 - Multiplicity: 1 teacher teaches many students → 1..*

N-ary Association (Short)

- **Definition:**
 - An **N-ary association** is a relationship **involving more than two classes**.
 - Used when a simple binary association is not enough to describe the relationship.
- **Example:**
 - Classes: Student, Course, Instructor
 - N-ary association: Enrollment connecting **all three classes**
 - Shows **which student is enrolled in which course with which instructor**
- **Notation:**
 - Draw a **diamond in the center**, connect **all participating classes** to it

Q. 7 What is state? Explain with example transitions and condition.

5 marks

Ans: -

State – Definition & Explanation

Definition:

- A **state** is a **condition or situation of an object/system at a particular time**, during which it **satisfies some condition, performs an activity, or waits for an event**.
- **States show how an object behaves over time.**

Key Concepts

Transition:

10. Transition:

- A **transition** is a **change from one state to another**.
- It happens when an **event occurs** that causes the object/system to move to a new state.
- **Notation:** Arrow from source state → target state
- **Example:**

[Idle] -- Insert Card --> [Card Inserted]

- **Explanation:** The system moves from Idle state to Card Inserted state when the user inserts a card.

11. Event / Trigger:

- The **event** is the **occurrence that initiates the transition**.
- **Example:** Insert Card triggers the transition from Idle → Card Inserted.
- **Explanation:** Events are the “stimuli” that the system responds to.

12. Condition / Guard:

SPPU TE BE COMP [Click to join telegram](#)

- A **condition or guard** is a **boolean expression that must be true** for the transition to occur.
- **Notation:** Written in square brackets [] on the transition.
- **Example:**

[PIN Correct] --> [Access Granted]
[PIN Incorrect] --> [Retry / Cancel]

- **Explanation:** Even if the event occurs, the transition only happens when the **condition is satisfied**.

Example – ATM System State Diagram

[Idle] -- Insert Card --> [Card Inserted]

[Card Inserted] -- Enter PIN --> [Authentication]

[Authentication] -- [PIN Correct] --> [Access Granted]

[Authentication] -- [PIN Incorrect] --> [Retry / Cancel]

Q. 8 Explain following:

5 marks

7. Composite state

8. Self transition orthogonal states with respect to state chart diagram.

Ans:-

Composite State

- **Definition:**
 - A **composite state** is a **state that contains one or more nested sub-states** inside it.
 - It is used to **group related states together** and show **complex behavior hierarchically**.
- **Purpose:**
 - Simplifies large state diagrams
 - Represents **hierarchical behavior**
- **Example:**
 - Authentication in ATM system is a composite state containing:
 - Enter PIN
 - Verify PIN
- **Notation/ Diagram:**

```
+-----+
| Authentication | <-- Composite State
| +-----+ /
| | Enter PIN   | | <-- Sub-state
| +-----+ /
| | Verify PIN  | | <-- Sub-state
| +-----+ /
+-----+
```

Self-Transition

- **Definition:**
 - A **self-transition** is a transition where a **state transitions to itself** in response to an event.
- **Purpose:**
 - Represents an **internal change or repeated activity** without leaving the state
- **Example:**
 - In Idle state, pressing **Cancel** repeatedly → stays in Idle
- **Notation/ Diagram :**

```
+-----+
| Idle   |
|        |
| ⤴ Event | <-- Loops back to itself
+-----+
```

Orthogonal States

- **Definition:**
 - **Orthogonal states** are **independent regions within a composite state** that can be **active simultaneously**.
 - Used to model **parallel or concurrent behavior**
- **Example:**
 - In Railway Reservation System:
 - Region 1 → Seat Selection
 - Region 2 → Payment Processing
 - Both regions active at the same time inside a composite state Reservation
- **Notation/ Diagram:**

```
+-----+
| Reservation | <-- Composite State
| +-----+ +-----+ /
| | Seat      | | Payment | | <-- Orthogonal / parallel regions
| | Selection | | Processing| |
| +-----+ +-----+ /
+-----+
```

Q. 9 Explain terms:

5 marks

7. Aggregation

8. Composition

Ans:-

Aggregation

- **Definition:**
 - Aggregation is a **“Has-A” relationship** where one class **contains another class** as a part, but **both can exist independently**.
 - It represents a **weak ownership**.
- **Notation:**
 - A **hollow diamond** at the end of the association line near the **whole** class.
- **Example:**
 - Library aggregates Book →
 - Library has many books
 - Books can exist even if Library is deleted
- **Diagram:**

Library ◇—— Book

Composition

- **Definition:**
 - Composition is a **strong “Has-A” relationship** where one class **owns another class**, and **the part cannot exist without the whole**.
 - It represents **strong ownership**.
- **Notation:**
 - A **filled diamond** at the end of the association line near the **whole** class.
- **Example:**
 - House composes Room →
 - Rooms cannot exist without the House
 - If House is deleted, all Rooms are deleted too
- **Diagram :**

House ◆—— Room

Q. 10 Explain different components of a state diagram

5 marks

Ans:-

Components of a State Diagram

A **state diagram** (or state chart diagram) is used to **model the dynamic behavior of a system or object**, showing **how it moves from one state to another in response to events**.

The main components are:

State

- Represents a **condition or situation of an object** during which it **performs activities or waits for events**.
- **Example:** Idle, Authentication, Access Granted in an ATM system.

Initial State

- The **starting point** of the state diagram.
- **Notation:** A **filled black circle**.
- **Example:** System starts in Idle state.

Final State

- Represents the **end point** of the process or object lifecycle.
- **Notation:** A **filled black circle inside a hollow circle**.
- **Example:** ATM transaction ends after Access Granted or Cancel.

Transition

- **Change from one state to another** triggered by an **event**.
- **Notation:** Arrow from source state → target state.
- **Example:** [Idle] -- Insert Card --> [Card Inserted]

Event / Trigger

- **Stimulus that causes a transition** between states.
- **Example:** Insert Card, Enter PIN, PIN Correct.

Condition / Guard

- A **boolean expression** that must be true for a transition to occur.
- **Notation:** Written in square brackets [].
- **Example:** [PIN Correct] → Access Granted

Action (Optional)

- **Activity performed** as a result of a transition.
- **Notation:** Written after “/” on the transition.
- **Example:** [PIN Correct] / Dispense Cash